

Reporte: Actividad 1 - Diseño de una ALU Combinacional

Curso: Diseño de Sistemas Digitales

Aguilar Andrade Ángel De Jesús

1. Resumen

En el presente documento se detalla el diseño, simulación e implementación de una Unidad Aritmético Lógica (ALU) de 4 bits utilizando exclusivamente circuitos combinacionales en SystemVerilog. La ALU desarrollada es capaz de realizar ocho operaciones distintas (Suma, Resta, AND, OR, XOR, NOT, Shift Left y Shift Right), seleccionables a través de un bus de control de 3 bits. Además, cuenta con cuatro banderas o "flags" (*negative*, *zero*, *overflow* y *carry/borrow*) para indicar el estado de cada operación aritmética o lógica. El sistema fue simulado y preparado para su implementación en la tarjeta de desarrollo BASYS3 de Digilent.

2. Introducción

Las Unidades Aritmético Lógicas son componentes fundamentales en la arquitectura de los procesadores y microcontroladores, ya que se encargan de ejecutar las operaciones matemáticas y lógicas sobre los datos. El diseño de una ALU a nivel combinacional permite comprender el enrutamiento de datos y la evaluación lógica sin la necesidad de señales de reloj (elementos secuenciales). El objetivo de esta actividad es aplicar los conocimientos en diseño lógico y SystemVerilog para construir una ALU paramétrica, verificando su comportamiento a través de un testbench (simulación funcional) y asignando pines (XDC) correspondientes para probarla físicamente en una FPGA.

3. Diseño General

El sistema central (*top-level*) consiste en un único módulo llamado `alu` cuyas entradas son:

- **A [3:0]**: Primer operando de 4 bits.
- **B [3:0]**: Segundo operando de 4 bits.
- **Operation [2:0]**: Selector de operación (8 combinaciones).

Las salidas del sistema son:

- **Output [3:0]**: Resultado de la operación seleccionada de 4 bits.
- **n**: Bandera de resultado negativo (bit más significativo del resultado).
- **z**: Bandera de cero (se activa si el resultado de salida es exactamente igual a 0).
- **o**: Bandera de desbordamiento u *overflow* (para sumas y restas con signo).
- **c**: Bandera de acarreo (*carry*) o préstamo (*borrow*).

El diseño interno se basa en una sentencia `always_comb` con una estructura `case` que enruta el resultado en base a `Operation`. Se extienden los operandos A y B a 5 bits temporalmente para evaluar correctamente el acarreo y préstamo de manera directa.

Operaciones de la ALU

Operation	Output	Description
000	$A + B$	Suma
001	$A - B$	Resta
010	$A \& B$	Bitwise AND
011	$A B$	Bitwise OR
100	$A \wedge B$	Bitwise XOR
101	$\sim A$	Bitwise NOT sobre A
110	$A \ll 1$	Desplazamiento Izq.
111	$A \gg 1$	Desplazamiento Der.

4. Análisis de Resultados (Simulación)

El sistema fue sometido a distintas pruebas (*testbench*) donde se verificaron las condiciones críticas de las operaciones:

- Suma (000):** Al sumar $A = 0111$ (7) y $B = 0001$ (1), se obtuvo $\text{Output} = 1000$ (-8). La bandera de *overflow* (o) se activó correctamente indicando que el límite de los números con signo se superó, así como la bandera n debido a que el bit más significativo resultó en 1.
- Resta (001):** En la operación $A = 0010$ (2) menos $B = 0101$ (5), el resultado fue $\text{Output} = 1101$ (-3), y las banderas n (negativo) y c (*borrow*) se activaron de forma correcta demostrando un préstamo virtual en el circuito.
- Desplazamiento Lógico (110 / 111):** El *shift left* evaluó el corrimiento de bits hacia la izquierda. Al tener $A = 1011$ y desplazar a la izquierda, la salida fue 0110 con la bandera de acarreo ($c = 1$), correspondiendo al MSB perdido. En *shift right* para $A = 0001$, la salida fue 0000 comprobándose también la activación de la bandera Zero ($z = 1$).
- Compuertas Lógicas:** Todas las operaciones bit a bit (AND, OR, XOR, NOT) funcionaron conforme a las tablas de verdad de forma consistente, sin alteración en el hardware físico de las banderas (con excepción de n o z cuando aplique).

5. Conclusiones

Se logró diseñar con éxito una ALU combinacional en SystemVerilog logrando integrar los operadores básicos aritméticos y lógicos soportados nativamente por el lenguaje. Al utilizar `always_comb`, se previó la inferencia indeseada de *latches* al establecer valores predeterminados de las señales antes del bloque `case`. Las banderas resultan determinantes para la toma de decisiones (saltos condicionales) en unidades procesadoras más robustas y se demostró que pueden resolverse con matemática de complemento a dos a nivel de hardware.

Apéndices

Código SystemVerilog (alu.sv)

```
`timescale 1ns / 1ps

module alu (
    input  logic [3:0] A,
    input  logic [3:0] B,
    input  logic [2:0] Operation,
    output logic [3:0] Output,
    output logic      n, // negative
    output logic      z, // zero
    output logic      o, // overflow
    output logic      c  // carry/borrow
);

    logic [4:0] A_ext;
    logic [4:0] B_ext;
    logic [4:0] res_ext;

    always_comb begin
        A_ext = {1'b0, A};
        B_ext = {1'b0, B};
        res_ext = 5'b00000;
        Output = 4'b0000;
        n = 1'b0; z = 1'b0; o = 1'b0; c = 1'b0;

        case (Operation)
            3'b000: begin
                res_ext = A_ext + B_ext;
                Output = res_ext[3:0];
                c = res_ext[4];
                o = (A[3] == B[3]) && (Output[3] != A[3]);
            end
            3'b001: begin
                res_ext = A_ext - B_ext;
                Output = res_ext[3:0];
                c = res_ext[4];
                o = (A[3] != B[3]) && (Output[3] != A[3]);
            end
        endcase
    end
endmodule
```

```
        end
        3'b010: Output = A & B;
        3'b011: Output = A | B;
        3'b100: Output = A ^ B;
        3'b101: Output = ~A;
        3'b110: begin
            Output = A << 1;
            c = A[3];
        end
        3'b111: Output = A >> 1;
        default: Output = 4'b0000;
    endcase

    if (Output == 4'b0000) z = 1'b1;
    else z = 1'b0;

    n = Output[3];
end
endmodule
```

Bibliografía

- [1] S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3ra ed. New York, NY, USA: McGraw-Hill, 2014.
- [2] Digilent, "Basys 3 FPGA Board Reference Manual". Accedido el: 30 de septiembre de 2026. [En línea]. Disponible en: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>